GA GUARDIAN

Protocol Review

Trueo

Security Assessment

April 23rd, 2026

Summary

Audit Firm Guardian

Prepared By Robert Regeida, 0xCiphky, Cryptic Defense

Client Firm Trueo

Final Report Date April 23, 2026

Audit Summary

Trueo engaged Guardian to review the security of their Staking PR. From the 8th through the 10th of April, a team of 3 auditors reviewed the source code in scope. All findings have been recorded in the following report.

Confidence Ranking

Given the minimal High and Critical issues detected during the main review, Guardian assigns a Confidence Ranking of 4 to the protocol. Guardian advises the protocol to consider periodic review with future changes.

For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

Note: Fixes to the findings uncovered in the Round 4 review have not been reviewed by Guardian. Guardian recommends the client either perform an amended follow-up review period to focus on the fixes to remediations findings or seek a follow-up audit from another team.

✅ Verify the authenticity of this report on Guardian's GitHub: https://github.com/guardianaudits

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. **Recommendation:** Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. **Recommendation:** Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. **Recommendation:** Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. **Recommendation:** Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. **Recommendation:** Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Overview

Project Summary

Project Name	Trueo
Codebase	https://github.com/truth-market/truth-contracts
Commits	Main Review: e88962cd2475e9bfecf97d39b19dc7d5a7d8a3a7 Remediation: c8d20002b366b8e25a754443dc942c64153bc344

Audit Summary

Delivery Date	April 23, 2026
Audit Methodology	Static Analysis, Manual Review

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
Critical	0	0	0	0	0	0
High	4	0	0	0	0	4
Medium	22	0	0	4	0	18
Low	23	0	0	6	0	17
Info	19	0	0	7	1	11

Audit Scope & Methodology

In-scope files:

```
truth-contracts/src/Launchpad.sol
truth-contracts/src/LaunchpadFactory.sol
truth-contracts/src/TruthMarketManager.sol
truth-contracts/src/TruthMarketV2Launcher.sol
truth-contracts/src/TruthMarketV2ProportionalDistributor.sol
truth-contracts/src/TruthMarketV2SingleSideLiquidityPositionDistributor.sol
truth-contracts/src/types/LaunchpadHook.sol
truth-contracts/src/types/LaunchpadProposal.sol
truth-contracts/src/types/LaunchpadTruthMarketV2.sol
truth-contracts/src/base/LaunchpadState.sol
truth-contracts/src/base/LaunchpadViewer.sol
truth-contracts/src/interfaces/IDistributor.sol
truth-contracts/src/interfaces/ILauncher.sol
truth-contracts/src/interfaces/ILaunchpad.sol
truth-contracts/src/interfaces/ILaunchpadFactory.sol
truth-contracts/src/interfaces/ILaunchpadViewer.sol
truth-contracts/src/interfaces/IPlugin.sol
truth-contracts/src/libraries/LaunchpadFactoryCommunityV2.sol
truth-contracts/src/libraries/LaunchpadFactoryMarketV2.sol
truth-contracts/src/libraries/LaunchpadFactoryWhalesV2.sol
truth-contracts/src/libraries/Roles.sol
truth-contracts/src/libraries/UniswapV4LPHelper.sol
truth-contracts/src/plugins/RestrictDepositorPlugin.sol
truth-contracts/src/plugins/RestrictMaximumRatioOfOutcomeDepositPlugin.sol
truth-contracts/src/plugins/RestrictMinimumDepositsPlugin.sol
truth-contracts/src/plugins/RestrictOutcomePlugin.sol
```

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: *High*	Impact: *Medium*	Impact: *Low*
Likelihood: *High*	🔴 Critical	🟠 High	🟡 Medium
Likelihood: *Medium*	🟠 High	🟡 Medium	🟢 Low
Likelihood: *Low*	🟡 Medium	🟢 Low	🟢 Low

Impact

High Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.

Medium A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.

Low Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

High The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.

Medium An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.

Low Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts. Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Round One Findings & Resolutions

ID	Title	Category	Severity	Status
H-01	Permissionless Markets Can Spend Global Rewards	Unexpected Behavior	High	Resolved
H-02	Pool Initialization Front-Run Bricks Launches	Frontrunning	High	Resolved
H-03	Premature Depositor Removal Loses Funds	Logical Error	High	Resolved
M-01	Unbounded Depositor Set Can Block Distribution	DoS	Medium	Resolved
M-02	Module Whitelist Checked Only At Prop. Creation	Unexpected Behavior	Medium	Resolved
M-04	Unvalidated Cap Can Stall Automated Distribution	DoS	Medium	Resolved
M-05	Deferred Distribution Failure Can Forfeit Retry	Unexpected Behavior	Medium	Resolved
M-06	Blacklist Bypass Via LaunchpadFactory	Validation	Medium	Resolved
M-07	Launch Readiness Not Re-evaluated On Withdraw	Logical Error	Medium	Resolved
M-08	Factory Minimum Deposit Hard-codes TYD Decimals	Unexpected Behavior	Medium	Resolved
M-09	Deferred Distribution Is Externally Manipulable	Logical Error	Medium	Resolved
M-10	Single-side LP Range Can Collapse Near 1.0	DoS	Medium	Acknowledged
M-11	Hardcoded Trading Window Drift	DoS	Medium	Resolved

Round One Findings & Resolutions

ID	Title	Category	Severity	Status
M-12	Gas-Griefed Distribution Forces Deferred Window	Gas Griefing	Medium	Acknowledged
M-13	Deposit-Only Min Check Allows Dust Contributions	Validation	Medium	Resolved
L-01	Failed-phase Withdraw Hooks Can Block Refunds	Unexpected Behavior	Low	Resolved
L-02	Zero-amount Withdraw Mutates State	Unexpected Behavior	Low	Resolved
L-03	Unbounded Hook Args Can Gas-grief Proposals	DoS	Low	Resolved
L-04	Default Ratio Guard Doesn't Prevent InvalidPrice	Rounding	Low	Resolved
L-05	Insufficient Reward Wallet Funds Bricks Launch	Validation	Low	Acknowledged
L-06	Withdrawal Lacks Slippage Protection	Validation	Low	Acknowledged
L-07	Community Spec Getter Can Return False Positive	Unexpected Behavior	Low	Resolved
L-08	Unsettled Market Can Block OracleBonds Rotation	DoS	Low	Acknowledged
L-09	Non-binary Outcomes Can Lock Deposits	Validation	Low	Resolved
L-10	Raw Approve Breaks Non-Standard Tokens	Compatibility	Low	Acknowledged
L-11	Dispute Bond Taken Silently During Failed Check	Logical Error	Low	Resolved

Round One Findings & Resolutions

ID	Title	Category	Severity	Status
I-01	Derived Failed Phase Is Not Persisted On-chain	Unexpected Behavior	Info	Acknowledged
I-02	Reentrancy Vector Via Custom Reward Token	Warning	Info	Resolved
I-03	Slippage Tolerance Exceeds Permit2 Approval	Validation	Info	Resolved
I-04	Last Depositor Bears Disproportionate Gas Cost	Suggestion	Info	Acknowledged
I-05	Stored Specs Drift From Applied Defaults	Unexpected Behavior	Info	Resolved
I-06	Initialize Allows Protocol Fee Above Fee Scale	Validation	Info	Resolved
I-07	Missing Market Payment-token Check	Validation	Info	Resolved
I-08	Active Markets Set Grows Unboundedly	Gas Optimization	Info	Acknowledged
I-09	Bond Amounts Snapshot At Market Creation	Documentation	Info	Resolved
I-10	Single-side LP Helper Allows Zero Liquidity	Unexpected Behavior	Info	Resolved
I-11	Unbounded Ratio Threshold Can Disable Check	Validation	Info	Resolved
I-12	Max Sqrt Clamp Can Break Tick Conversion	Validation	Info	Resolved
I-13	Low Whales Ratio Can Deadlock Launch	Validation	Info	Resolved

Round One Findings & Resolutions

ID	Title	Category	Severity	Status
I-14	Single-side Distributor Reuses Slot0/tick	Unexpected Behavior	Info	Acknowledged
I-15	Incompatibility With Fee On Transfer Tokens	Warning	Info	Acknowledged
I-16	Proposal Creator Metadata Is Spoofable	Warning	Info	Resolved

H-01 | Permissionless Markets Can Spend Global Rewards

Category	Severity	Location	Status
Unexpected Behavior	High	LaunchpadFactory.sol, TruthMarketManager.sol	Resolved

Description

The permissionless launch flow allows any user to submit market parameters through `LaunchpadFactory`, including `rewardToken` and `rewardAmount`. Those values are forwarded through `TruthMarketV2Launcher` into `TruthMarketManager.createMarketV2` without a policy gate that binds rewards to trusted proposers, per-market caps, or pre-funded creator escrow. When a market is created, `TruthMarketManager` pulls rewards from the shared `rewardWallet` using `safeTransferFrom(rewardWallet, marketAddress, rewardAmount)`. This means reward spending authority is delegated to any actor who can create and launch a proposal, as long as the manager has allowance from the reward wallet for the selected token. Because factory proposal creation is externally callable and the factory is granted proposer privileges in deployment config, this becomes a permissionless path to consume global reward inventory. An attacker can repeatedly create launchable markets with arbitrary reward values and force transfers from the treasury-configured reward source into attacker-chosen markets, draining or locking...

Recommendation

Move reward funding from global wallet pulls to per-proposal escrow supplied by the proposal creator or a designated sponsor. If global rewards must remain supported, enforce strict governance controls on reward-bearing proposals by requiring trusted proposers, whitelisted reward tokens, and hard per-market and per-epoch limits. Add explicit validation for reward parameter coherence and reject unsupported reward sources by policy instead of relying on wallet allowance side effects.

Resolution

Trueo: Resolved.

H-02 | Pool Initialization Front-Run Bricks Launches

Category	Severity	Location	Status
Frontrunning	High	TruthMarketV2Launcher.sol: 103-104	Resolved

Description

When a deposit triggers a successful launch, `TruthMarketV2Launcher.launch()` creates the market via `TruthMarketManager`, then initializes both Uniswap V4 pools:
Which calls `poolManager.initialize(poolKey, sqrtPriceX96);`. Uniswap V4's initialize reverts if the pool already exists. The `TruthMarketHook` does not protect against this, as it sets `beforeInitialize: false`. An attacker who observes a launch triggering within `deposit()` can derive the resulting token addresses and pool keys, then front-run with `poolManager.initialize(poolKey, anySqrtPrice)` for either pool. This attack can be utilized to block any proposal launch. Users must wait until `block.timestamp > endTime` to withdraw without a fee.

Recommendation

Enable `beforeInitialize` on `TruthMarketHook` and restrict pool initialization to the launcher contract.

Resolution

Trueo: Resolved.

H-03 | Premature Depositor Removal Loses Funds

Category	Severity	Location	Status
Logical Error	High	TruthMarketV2ProportionalDistributor.sol: 57	Resolved

Description

When a user withdraws their full balance from a single outcome of a proposal, `withdraw()` removes them from the global depositors `EnumerableSet` even if they hold deposits in another outcome.
A user may have deposited to both outcomes, for example to strategically build a balanced launch position rather than a pure directional bet. If a user withdraws their full `balance = deposits[msg.sender][outcome]` of an outcome, `removeDepositor` removes the user from the set the `distributor` relies on to process depositor outcome: `address[] memory depositors = viewer.proposalDepositors(proposalId);` A user who deposits to outcomes 1 and 2, then fully withdraws from outcome 1, is removed from the set while still holding funds in outcome 2. On launch, their remaining deposit inflates the mint amount but they receive nothing. Post-launch withdrawal is blocked due to the `phase` check, so funds are permanently lost.

Recommendation

Track depositor outcome count and only remove from the set when all outcome balances are zero:

Resolution

Trueo: Resolved.

M-01 | Unbounded Depositor Set Can Block Distribution

Category	Severity	Location	Status
DoS	Medium	TruthMarketV2ProportionalDistributor.sol, TruthMarketV2SingleSideLiquidityPositionDistributor.sol	Resolved

Description

The distribution flow is designed as a single-shot operation over the full depositor set, but depositor growth is not bounded. During deposits, every participant is added to an enumerable set that can keep growing until launch conditions are met.
At settlement time, the distributors fetch the entire depositor list into memory and iterate across all participants in one transaction. The proportional distributor performs two outcome passes per depositor, while the single-side distributor also performs LP-position work inside the loop. Gas cost therefore scales with participant count before distribution can complete.
`Launchpad` attempts distribution atomically. If the distributor call becomes too expensive and fails, the proposal is deferred or pushed into manual distribution handling. In practice, a sufficiently large depositor set can turn automatic settlement into an operator-dependent recovery path.

Recommendation

Replace one-shot full-set distribution with paginated settlement that persists progress on-chain. Snapshot settlement inputs once, process bounded depositor ranges per transaction, and finalize only after the last batch succeeds.
```solidity
function distributeRange(
ILaunchpadViewer viewer,
uint256 proposalId,
uint256 start,
uint256 maxDepositors
) external returns (uint256 processed, bool done);
```

Resolution

Trueo: Resolved.

M-02 | Module Whitelist Checked Only At Prop. Creation

Category	Severity	Location	Status
Unexpected Behavior	Medium	Launchpad.sol, LaunchpadProposal.sol	Resolved

Description

The module whitelist is enforced when a proposal is created, but launch and distribution later execute using stored module addresses without re-checking current whitelist status. This creates a policy gap where a proposal can continue to use a launcher, distributor, or hook plugin even after that module has been removed from the whitelist. At proposal creation, the protocol validates `proposal.launcher`, `proposal.distributor`, and each bound plugin against `_moduleWhitelist`.
After creation, execution paths consume the stored module addresses directly. The launch path calls `state.proposal.launcher` and the distribution path calls `state.proposal.distributor`, with no whitelist gate at those boundaries.
Because whitelist status is not re-evaluated at execution time, de-whitelisting a module does not reliably prevent already-created proposals from invoking it. This weakens the expected effect of emergency module disablement and can preserve exposure to modules that policy intended to block.

Recommendation

Enforce whitelist checks at execution boundaries in addition to proposal creation. Before calling launcher, distributor, or proposal-bound plugins, require that each module is still whitelisted. This aligns runtime behavior with current policy and makes emergency de-whitelisting effective immediately.
Apply this guard in launch and distribution flows before external calls, and before plugin hook execution. If repeated plugin scans are too expensive, snapshot an immutable `allowedModulesHash` at...

Resolution

Trueo: Resolved.

M-04 | Unvalidated Cap Can Stall Automated Distribution

Category	Severity	Location	Status
DoS	Medium	LaunchpadFactoryMarketV2.sol	Resolved

Description

TruthMarketV2 cap is passed through from proposal params to market creation and affects whether distribution succeeds, but proposal validation does not enforce a minimum-viable cap against launch-time mechanics. Factory-side common validation contains only question + timing checks: That path does not enforce constraints on params.cap before launch. The cap is consumed as yesNoTokenCap when market creation/minting occurs:
During launch distribution:
If market minting later fails (e.g., from insufficient remaining mint capacity due to a too-low cap), the automated path can move into deferred/manual recovery mode. In practice this creates a failure mode with no user-controlled fallback; completion depends on resolver intervention. Because of this proposals can become launchable/accepted by factory checks yet be settlement-fragile: • normal distribution reverts at launcher/distributor boundary, • phase transitions skip to deferred/manual handling, • users lose deterministic, permissionless settlement expectation.

Recommendation

Add cap invariants during proposal validation and launch-time readiness checks: • require cap > 0, • require cap >= protocol-fee-adjusted minimum expected distributable, • ensure distributable projection at proposal level cannot exceed remaining token mint capacity before setting proposal ready. Fail fast in validateCommon or early launch check with clear revert reasons if cap cannot satisfy expected max mint for this proposal.

Resolution

Trueo: Resolved.

M-05 | Deferred Distribution Failure Can Forfeit Retry

Category	Severity	Location	Status
Unexpected Behavior	Medium	Launchpad.sol	Resolved

Description

`resolveDeferredDistribution()` invokes `_distribute(..., true)` when a launched proposal is not yet distributed:
Inside `_distribute`, any distributor error during deferred resolution sets the phase to `ManualDistributionNeeded`:
Because `resolveDeferredDistribution()` is gated on `ProposalPhase.Launched`, once this catch branch executes there is no path back to retry `_distribute` automatically; the proposal is forced into manual recovery mode. Failures during one deferred attempt are treated as terminally unrecoverable in protocol logic, even though they could be retryable. The main impacts are: • Non-deterministic failures can be converted into permanent manual-recovery outcomes. • A resolver can be forced into custodial `withdrawFunds` flow by inducing a single failing attempt. • This weakens liveness and increases trust in privileged resolver actions.

Recommendation

Keep a retryable deferred state and allow repeated attempts after transient failure, only escalating to manual custody after bounded failures or explicit governance approval. Suggested pattern: • Add failure counter and/or cool-down window for deferred resolution attempts. • Remain in `ProposalPhase.Launched` for retryable failures. • Move to `ManualDistributionNeeded` only after N failed attempts or explicit admin decision.

Resolution

Trueo: Resolved.

M-06 | Blacklist Bypass Via LaunchpadFactory

Category	Severity	Location	Status
Validation	Medium	Launchpad.sol: 124	Resolved

Description

Launchpad.propose enforces blacklist checks against msg.sender, but in factory flows msg.sender at Launchpad is the LaunchpadFactory contract, not the end user. A blacklisted user can call proposeCommunityTruthMarketV2 or proposeWhalesTruthMarketV2 through the factory and still create proposals as long as the factory has proposer permissions, which bypasses blacklist enforcement for proposal creation.

Recommendation

In Launchpad.propose, enforce blacklist validation on proposal.creator instead of msg.sender.

Resolution

Trueo: Resolved.

M-07 | Launch Readiness Not Re-evaluated On Withdraw

Category	Severity	Location	Status
Logical Error	Medium	Launchpad.sol: 227	Resolved

Description

Launchpad._attemptLaunch is only invoked from deposit, while withdraw also changes the exact state. This means a proposal can become launch-ready after a withdrawal (for example, max-ratio becomes compliant) but remain stuck in Live until another deposit occurs. If no further deposit happens before endOfProposal, the proposal can expire to Failed even though launch conditions were satisfied after the withdrawal.

Recommendation

Re-evaluate launch conditions after withdraw, similar to how deposit invokes _attemptLaunch.

Resolution

Trueo: Resolved.

M-08 | Factory Minimum Deposit Hard-codes TYD Decimals

Category	Severity	Location	Status
Unexpected Behavior	Medium	LaunchpadFactoryWhalesV2.sol, LaunchpadFactoryCommunityV2.sol, TruthMarketManager.sol	Resolved

Description

The factory `minimumDeposit` checks for both Community and Whales proposals are hard-coded against fixed raw units that encode a TYD/6-decimal assumption, while the active payment token is protocol-configured and not fixed in the factory. Community path:
Whales path:
Both proposals flow into market creation against the protocol payment token configured in `TruthMarketManager`:
Because validation compares raw units, this creates: • Wrongly strict floor for low-decimal environments where the literal assumes too much scale, • Trivially weak floor for high-decimal environments where the same raw bound represents far less than intended economic value. Thus the protocol's economic guard is token-decimal-dependent but tied to protocol config, not to actual token decimals.

Recommendation

Normalize/check `minimumDeposit` against the configured payment token decimals (or stable fixed unit policy) in both factory paths, e.g.: • `minimumDeposit >= MIN_BASE * 10**paymentToken.decimals()`. Avoid literal assumptions in factory-level protocol checks unless the token is permanently fixed.

Resolution

Trueo: Resolved.

M-09 | Deferred Distribution Is Externally Manipulable

Category	Severity	Location	Status
Logical Error	Medium	Launchpad.sol: 294	Resolved

Description

The launch flow deploys TruthMarketV2 and initializes both pools, then transitions the proposal to Launched before calling the distributor. If distribution reverts, the proposal remains Launched and allocation is retried later, while the market and pools are already live and externally mutable. In this launched-but-undistributed window, attackers can manipulate state that deferred allocation depends on. The yesNoTokenCap headroom can be consumed through external minting so distributor-required mint capacity is no longer available, preventing completion. An attacker can also shift the pool tick before deferred allocation so single-sided LP bounds are derived from a manipulated live tick state and collapse into an invalid range where tickLower >= tickUpper. The same window also enables a sandwich around deferred allocation where the attacker moves tick before allocation, the protocol places liquidity using that manipulated state, and the attacker back-runs after allocation to obtain better execution for the same fixed input.

Recommendation

Make launch and distribution atomic so the proposal cannot remain in a launched-but-undistributed state with live mutable market/pool state.

Resolution

Trueo: Resolved.

M-10 | Single-side LP Range Can Collapse Near 1.0

Category	Severity	Location	Status
DoS	Medium	UniswapV4LPHelper.sol	Acknowledged

Description

This protocol lets users bet on binary outcomes (for example YES / NO): 1. Users deposit payment tokens into a proposal on the Launchpad. 2. When launch conditions are met, the proposal is launched and a Uniswap V4 market is created for that question. 3. On settlement, the distributor sends users their outcome tokens and creates LP positions in both outcome pools. The LP step for this protocol path uses: • TruthMarketV2SingleSideLiquidityPositionDistributor.distribute(...) • UniswapV4LPHelper.calculateSingleSidedPositionParams(...) The helper precomputes a tick range and reuses it for all users in the proposal. The protocol's fairness intent is purely proportional math from recorded deposits:
Meaning: • If a bettor put in depositAmount for an outcome and that outcome has totalDeposits, their payout share is depositAmount / totalDeposits. • This share is multiplied by totalDistributable to get their outcome payout amount. • Same formula runs for everyone; no manual choice in allocation. The implementation processes both outcomes with pre-fetched pool data:

Recommendation

Compute one-sided LP bounds from live state in a way that guarantees non-zero width across boundary cases, add an explicit precondition that enforces tickLower < tickUpper before LP creation and add launch-time or proposal-validation checks that reject configurations (including problematic tickSpacing values) where single-sided math can collapse at initialization; if distribution remains long-running, switch to bounded/batch settlement with drift checks rather than relying on one-shot...

Resolution

Trueo: Acknowledged.

M-11 | Hardcoded Trading Window Drift

Category	Severity	Location	Status
DoS	Medium	LaunchpadFactoryMarketV2.sol: 32	Resolved

Description

The launch flow validates endOfTrading against two different policies at two different stages. At proposal creation, LaunchpadFactoryMarketV2.validateCommon accepts any value where endOfTrading > endOfProposal + 1 hour (hardcoded). At launch execution, TruthMarketManager._createMarketCommon requires endOfTrading >= block.timestamp + minimumTradingDuration, where minimumTradingDuration is mutable via setDurations. This creates two independent constraints that can diverge over time. The launch process can therefore accept parameter values that are valid in factory checks but impossible under runtime manager checks, causing launch-triggering deposits to revert with InvalidEndOfTrading. If minimumTradingDuration is increased above the factory's fixed one-hour assumption, proposals can pass factory validation but fail at launch with InvalidEndOfTrading, creating launch-time liveness failures for otherwise accepted proposals. If minimumTradingDuration is reduced below one hour, the opposite drift remains: factory continues rejecting configurations that manager would allow, causing...

Recommendation

Replace the hardcoded +1 hour rule in factory validation with a minimum aligned to manager policy (for example, pass the manager-aligned minimum duration into library validation), so accepted proposal parameters are always launchable under the same constraints.

Resolution

Trueo: Resolved.

M-12 | Gas-Griefed Distribution Forces Deferred Window

Category	Severity	Location	Status
Gas Griefing	Medium	Launchpad.sol: 331-347	Acknowledged

Description

The last depositor who triggers launch controls the gas limit of their transaction. Due to the EIP150 63/64 gas rule, they can supply enough gas for launch() to succeed but starve distribute() inside the try/catch:
According to EIP150, an external call is allocated 63/64 of gas, leaving extra to complete the transaction if the call runs out of gas. The market is deployed and pools initialized, but distribution fails due to out of gas, and enters the catch block, enforcing a deferred distribution. The proposal is stuck in Launched, where depositors cannot withdraw. To resolve this, the LAUNCHPAD_RESOLVER_ROLE must manually call resolveDeferredDistribution(). During this window, the market is live with initialized pools but no depositor liquidity. **Note:** While the attacker can force a deferred distribution via gas griefing, pool manipulation during this window is not possible as it stands, since both pools are immediately paused after initialization and are only unpaused in finalize, which executes only after a successful distribute(). Any swap attempt during the...

Recommendation

Require a minimum gas threshold before entering the distribution path, or separate launch and distribution transactions so the launcher cannot be used to starve the distributor.

Resolution

Trueo: Acknowledged.

M-13 | Deposit-Only Min Check Allows Dust Contributions

Category	Severity	Location	Status
Validation	Medium	Launchpad.sol: 227	Resolved

Description

The documentation states that RestrictMinimumDepositsPlugin “prevents small deposits from being accepted, ensuring only meaningful contributions are allowed during the fundraising phase.” However, the implementation enforces this policy only during deposit and launch-threshold checks, not during withdrawals. Since the withdrawal path applies no equivalent minimum-stake retention rule, a user can satisfy the minimum contribution requirement, then withdraw almost all funds and leave a dust balance, bypassing the stated “meaningful contributions” guarantee in practice. This can also clog depositor tracking with economically meaningless positions, increasing overhead and gas costs for operations that iterate or process depositor state.

Recommendation

If the goal is to keep each participant’s contribution above a minimum threshold throughout escrow, enforce the same minimum on withdrawals by reverting any withdrawal that would leave a non-zero balance below the configured floor, unless it is a full withdrawal.

Resolution

Trueo: Resolved.

L-01 | Failed-phase Withdraw Hooks Can Block Refunds

Category	Severity	Location	Status
Unexpected Behavior	Low	Launchpad.sol	Resolved

Description

Launchpad.withdraw() permits withdrawals in both Live and Failed phases, but still executes all BeforeWithdraw hooks in both cases:
runBeforeWithdrawHooks iterates all BeforeWithdraw plugins and any reverting plugin blocks the call. Because of this: • A single misbehaving or intentionally reverting BeforeWithdraw hook can deny all refunds for failed proposals. • This can strand users in a state where they cannot retrieve escrowed funds, despite Phase == Failed. • The failure mode is especially severe because hook logic is external and can be upgraded/modified by module governance.

Recommendation

Skip BeforeWithdraw hooks for failed-proposal refunds (or maintain a strict allowlist of hooks that are refund-safe when phase == Failed). At minimum, guard plugin execution:
If failed-phase hooks are intentional by design, require explicit documentation and add a hard governance check that forbids reverting hooks from blocking standard refunds.

Resolution

Trueo: Resolved.

L-02 | Zero-amount Withdraw Mutates State

Category	Severity	Location	Status
Unexpected Behavior	Low	Launchpad.sol	Resolved

Description

`runBeforeWithdrawHooks` can execute arbitrary hook logic and a transfer side-effects follow this branch based on `amount`. If a caller has `balance == 0` in the target `outcome`, a call with any positive `amount` clamps to zero and: • emits a withdrawal event with `amount = 0`, • still mutates bookkeeping (`removeOutcome` if tracked outcome total reaches zero, `removeDepositor` when `balance == amount`), • and passes `amount=0` through hook + transfer path. In practice, this means users can accidentally or intentionally trigger removals and state transitions without economic effect, and plugins that assume non-zero `amount` at hook/runtime boundaries can mis-handle control flow.

Recommendation

After clamping and hook invocation, gate execution on a non-zero resolved amount:
or use a separate early-return path for no-op withdrawals that does not touch state/set bookkeeping. When removing depositor entries, require that all outcome balances are zero after mutation rather than relying solely on the original `balance`.

Resolution

Trueo: Resolved.

L-03 | Unbounded Hook Args Can Gas-grief Proposals

Category	Severity	Location	Status
DoS	Low	LaunchpadFactoryWhalesV2.sol, Launchpad.sol, LaunchpadHook.sol	Resolved

Description

`Launchpad.propose()` has no upper bound on `proposal.hookBindings.length` and no upper bound on per-hook payload size (`bytes args` in `HookBinding`):
Runtime paths iterate hooks linearly on every deposit/withdraw/check:
The direct `Launchpad.propose()` path is role-gated, but proposal creation is also permissionless through `LaunchpadFactory`:
In the whales flow, user-controlled `spec.whitelist` (unbounded length) is encoded into `RestrictDepositorPlugin` hook args:
`RestrictDepositorPlugin` decodes and linearly scans this whitelist on deposit and withdraw:
As a result, permissionless users can create proposals whose interaction cost is artificially high, causing repeated user transaction failures due to gas constraints. Attack vector: 1. Any user can call `proposeWhalesTruthMarketV2` (permissionless factory entrypoint). 2. The user supplies a very large whitelist in `spec.whitelist`. 3. The whitelist is persisted as hook args and evaluated on each deposit/withdraw. 4. Interaction gas grows with whitelist length and can exceed practical gas limits. 5. Refund withdrawals can...

Recommendation

Consider adding a strict whitelist length cap in whales spec validation (for example, `<= 128`), a hard cap on `proposal.hookBindings.length` in `Launchpad.propose()` and Add per-binding `args` byte-length caps and/or cap total encoded hook bytes per proposal. Finally, consider also adding a gas-safe emergency withdrawal path for failed proposals that bypasses expensive policy hooks.

Resolution

Trueo: Resolved.

L-04 | Default Ratio Guard Doesn’t Prevent InvalidPrice

Category	Severity	Location	Status
Rounding	Low	RestrictMaximumRatioOfOutcomeDepositPlugin.sol: 49	Resolved

Description

LaunchpadFactoryCommunityV2 and LaunchpadFactoryWhalesV2 rewrite maxRatioThreshold from 0 to 9999 to avoid one-sided launch failures, but the ratio hook and launch path use floored integer math in a way that is inconsistent with that goal. In RestrictMaximumRatioOfOutcomeDepositPlugin.checkProposal, each outcome ratio is computed as (outcomeDeposits * 10000) / totalDeposits and rejected only when ratio > maxRatioThreshold. In near one-sided states, this can produce 9999 for the dominant side and 0 for a non-zero minority side, so readiness returns true at threshold 9999. Launch then executes Launchpad._attemptLaunch -> TruthMarketV2Launcher.launch, where pool init prices are recomputed with the same floored formula. The minority side init price becomes 0, UniswapV4LPHelper.priceToSqrtPriceX96 reverts with InvalidPrice, and the launch attempt fails.

Recommendation

Add an explicit readiness guard in ratio check so proposals are not launch-ready when a non-zero side floors to zero (e.g., if (outcomeDeposits > 0 && ratio == 0) return false).

Resolution

Trueo: Resolved.

L-05 | Insufficient Reward Wallet Funds Bricks Launch

Category	Severity	Location	Status
Validation	Low	TruthMarketManager.sol: 823-826	Acknowledged

Description

When a proposal triggers launch, `TruthMarketManager._createMarketCommon()` transfers reward tokens from the shared `rewardWallet`:
If `rewardWallet` has insufficient balance at launch time (i.e., due to other markets draining the same token), the safeTransferFrom reverts, bricking the entire launch until the `rewardWallet` has sufficient funds again.

Recommendation

Validate reward token availability before market creation, or escrow reward tokens at proposal creation time.

Resolution

Trueo: Acknowledged.

L-06 | Withdrawal Lacks Slippage Protection

Category	Severity	Location	Status
Validation	Low	Launchpad.sol: 265-271	Acknowledged

Description

withdraw() calculates the payout using the current protocolFee at execution time with no minimum payout parameter:
If an admin increases the fee via setProtocolFee between a user's transaction submission and execution, the user will receive less funds than expected.

Recommendation

Add a minAmountOut parameter to withdraw:

Resolution

Trueo: Acknowledged.

L-07 | Community Spec Getter Can Return False Positive

Category	Severity	Location	Status
Unexpected Behavior	Low	LaunchpadFactory.sol	Resolved

Description

`LaunchpadFactory` stores community proposals in `_communitySpecs` and exposes `communityTruthMarketV2Spec(proposalId)` as a typed getter. The getter loads `spec = _communitySpecs[proposalId]` and then validates only `spec.proposalType == ProposalType.CommunityMarketV2`. Because `ProposalType.CommunityMarketV2` is the zero value of the enum, any unmapped `proposalId` returns the default zero-initialized struct and still passes the type check. As a result, callers can receive a community spec for an ID that was never created through `proposeCommunityTruthMarketV2`.

Recommendation

Track proposal existence explicitly and require it in the getter. A simple fix is to set a dedicated existence flag when persisting a spec and revert if the flag is false.

Resolution

Trueo: Resolved.

L-08 | Unsettled Market Can Block OracleBonds Rotation

Category	Severity	Location	Status
DoS	Low	TruthMarketManager.sol	Acknowledged

Description

`setOracleBonds` and `setOracleBondsWithBatchCheck` require every market in `_activeMarkets` to have `bondSettled() == true` before rotation succeeds. Because `_activeMarkets` grows over time and is not pruned for historical markets, a single unresolved market can block OracleBonds rotation indefinitely.

Recommendation

Decouple rotation from full historical market settlement. Keep an explicit set of currently blocking markets (or a removable unsettled index), and require settlement checks only for that set. Ensure settled/irrelevant markets are removed from the rotation gate so one stale market cannot deadlock upgrades.

Resolution

Trueo: Acknowledged.

L-09 | Non-binary Outcomes Can Lock Deposits

Category	Severity	Location	Status
Validation	Low	Launchpad.sol	Resolved

Description

`Launchpad` core deposit accounting permits arbitrary `outcome` values, and downstream launch/distribution logic only supports outcomes `1` and `2`. Core deposit path stores arbitrary outcome IDs without validation:
By contrast, launch and distribution logic is hard-coded to two outcomes only:
Funds deposited to any outcome other than `1` or `2` remain in `totalDeposits`, so launch price math and payouts are computed as if they do not exist, leaving that value unusable by the expected settlement rules.

Recommendation

Enforce output domain in the core deposit path (or require and verify enforced domain in proposal validation): • reject any `outcome` outside {1,2} at `Launchpad.deposit()`, or • make `launch/distribution` assert `proposalOutcomeDeposits(1)+proposalOutcomeDeposits(2)==proposalTotalDeposits` and revert if not. If additional outcome sets are intended, add full multi-outcome accounting and distribution support instead of assuming binary-only settlement downstream.

Resolution

Trueo: Resolved.

L-10 | Raw Approve Breaks Non-Standard Tokens

Category	Severity	Location	Status
Compatibility	Low	TruthMarketManager.sol: 644-650	Acknowledged

Description

`TruthMarketManager` uses raw `IERC20.approve()` when setting oracle bonds:
Solidity's ABI decoding expects `approve` to return bool. However, tokens like USDT (mainnet) return void, causing the decoder to revert on empty return data. Additionally, for tokens that do return bool, a false return on failure is silently ignored. Since `paymentToken` is admin-configurable, choosing USDT or similar non-standard tokens would brick `setOracleBonds` and `setOracleBondsWithBatchCheck`.

Recommendation

Use OpenZeppelin's SafeERC20 `forceApprove` method to handle non-standard tokens.

Resolution

Trueo: Acknowledged.

L-11 | Dispute Bond Taken Silently During Failed Check

Category	Severity	Location	Status
Logical Error	Low	TruthMarketManager.sol: 408-411	Resolved

Description

In `TruthMarketManager.disputeMarket()`, `sendDisputorBondToMarket()` is called unconditionally before the `ResolutionProposed` status check, which transfers the bound amount from the `disputor` to the `OracleBonds` contract. If the market is not in `ResolutionProposed` status, the disputor's tokens are taken but `raiseDispute()` is never called, and the function silently returns:
The disputor loses their bond with no dispute actually raised. Note that `escalateDisputeMarket()` does not have this issue as it calls `raiseEscalatedDispute()` unconditionally, which will revert if the status is invalid.

Recommendation

Move the status check before the bond transfer, and consider reverting on mismatch:

Resolution

Trueo: Resolved.

I-01 | Derived Failed Phase Is Not Persisted On-chain

Category	Severity	Location	Status
Unexpected Behavior	Info	LaunchpadViewer.sol	Acknowledged

Description

The `proposalPhase` view function derives `Failed` from time by comparing the current timestamp against `endTime` when the stored phase is still `Live`. The stored `state.phase` is only updated during mutating flows that invoke `_handlePhaseTransition` and the `ProposalFailed` event is emitted only from that transition path. This means an expired proposal can appear as `Failed` through `proposalPhase` while the persisted state remains `Live` and no failure event exists yet if no one calls a mutating function after expiry. This creates an observability consistency gap between derived read results and persisted lifecycle data. Systems that rely on events or raw storage state can lag behind systems that rely on `proposalPhase`.

Recommendation

Add an explicit finalization entrypoint that anyone can call after expiry to persist `Failed` and emit `ProposalFailed` deterministically when a proposal times out without launching. Keep this operation idempotent so repeated calls are safe and document that event-driven integrations should use the finalization flow when strict on-chain phase persistence is required.

Resolution

Trueo: Acknowledged.

I-02 | Reentrancy Vector Via Custom Reward Token

Category	Severity	Location	Status
Warning	Info	TruthMarketManager.sol: 823-826	Resolved

Description

A proposer can set `rewardToken` to a custom ERC20 with transfer hooks, creating a reentrancy entry point during `_createMarketCommon` (by pre-funding the rewardWallet with the token):
At this point, the market is deployed and initialized but pools are not yet initialized. The Launchpad's and TruthMarketManager's `nonReentrant` guards block reentry into all critical paths, and no exploitable vector was identified. However, future changes to the code between market creation and pool initialization could expose a reentrancy path.

Recommendation

Consider restricting `rewardToken` to a whitelist of known tokens, or moving the reward transfer to after pool initialization and token ownership transfer.

Resolution

Trueo: Resolved.

I-03 | Slippage Tolerance Exceeds Permit2 Approval

Category	Severity	Location	Status
Validation	Info	UniswapV4LPHelper.sol: 287-298	Resolved

Description

In `createSingleSidedPositionWithParams`, `amountMax` includes 0.5% slippage but `ensurePermit2Approval` only approves the base amount (without slippage applied):
If the full slippage were ever utilized, the `addLiquidity` operation would revert due to insufficient approval. Since all positions are created atomically within a single transaction where no price changes occur, the exact `tokenAmount` is consumed every time. However, it's important to note that if the slippage were ever utilized, distribution will always revert.

Recommendation

Consider documenting this or align the Permit2 approval with `amountMax`.

Resolution

Trueo: Resolved.

I-04 | Last Depositor Bears Disproportionate Gas Cost

Category	Severity	Location	Status
Suggestion	Info	Launchpad.sol: 224	Acknowledged

Description

The depositor whose deposit triggers launch conditions pays gas for the entire `launch` and `distribution` flow on top of their own deposit: market deployment, two pool initializations, and looping through all depositors to mint tokens and create LP positions. Every other depositor pays only for their deposit transaction. This creates a negative incentive where depositors may intentionally stay below the launch threshold to avoid being the trigger, stalling proposals despite near-sufficient deposits.

Recommendation

Consider separating launch triggering into a dedicated keeper call with gas incentives (e.g., a small bonus from the protocol fee), rather than relying on the last depositor.

Resolution

Trueo: Acknowledged.

I-05 | Stored Specs Drift From Applied Defaults

Category	Severity	Location	Status
Unexpected Behavior	Info	LaunchpadFactoryMarketV2.sol	Resolved

Description

Factory proposal composition uses two separate data paths. In `LaunchpadFactoryMarketV2.createBaseProposal`, an in-memory copy of market params is normalized first, including defaulting `fee` to `3000`, `tickSpacing` to `60`, and zeroing `rewardAmount` when `rewardToken` is zero, and then this normalized struct is encoded into `Proposal.params`. In parallel, `LaunchpadFactory.proposeCommunityTruthMarketV2` and `LaunchpadFactory.proposeWhalesTruthMarketV2` persist `_communitySpecs[proposalId] = spec` and `_whalesSpecs[proposalId] = spec` using the original user-provided spec without applying those normalized defaults. Because there is no write-back or reconciliation between these paths, spec retrieval endpoints can return values that differ from the effective parameters actually used at launch. This mismatch can mislead indexers, dashboards and reviewers that treat stored specs as source of truth for launched market configuration.

Recommendation

Persist the normalized spec after defaults are applied, or expose a dedicated view that returns effective launch parameters exactly as encoded into `Proposal.params`.

Resolution

Trueo: Resolved.

I-06 | Initialize Allows Protocol Fee Above Fee Scale

Category	Severity	Location	Status
Validation	Info	Launchpad.sol	Resolved

Description

The `Launchpad` initializer accepts `protocolFee_` without enforcing the same upper bound that is required later by `setProtocolFee`. This creates an inconsistent configuration surface where deployment can store a fee value that regular admin updates would reject.
Fee math later assumes bounded values when computing `(amount, fee)` from `rawAmount`.
If initialization sets `protocolFee` above `FEE_SCALE`, payout paths can produce zero distributable amounts for small values and can revert on subtraction for larger values. Impact is a misconfiguration-driven denial of expected fund flows until an admin correction is made.

Recommendation

Apply the same bound check in `initialize` that is already present in `setProtocolFee`, so invalid fee configuration is impossible at deployment time.

Resolution

Trueo: Resolved.

I-07 | Missing Market Payment-token Check

Category	Severity	Location	Status
Validation	Info	TruthMarketV2ProportionalDistributor.sol	Resolved

Description

`TruthMarketV2ProportionalDistributor` pulls `paymentToken` from its constructor, then calls `market.mint(distributable)` without checking that the target market uses the same payment token. If an inconsistent market/distributor pair is introduced (for example through factory/launcher misconfiguration, compromised module settings, or incorrect proposal wiring), the distributor can revert during mint or perform unintended external flow before any outcome token payout.

Recommendation

Add an explicit invariant before mint:
Keep this check as a hard precondition both at distributor entry and in any launcher path that selects paired modules.

Resolution

Trueo: Resolved.

I-08 | Active Markets Set Grows Unboundedly

Category	Severity	Location	Status
Gas Optimization	Info	TruthMarketManager.sol: 728	Acknowledged

Description

Markets are added to `_activeMarkets` on creation but never removed after finalization. This set grows indefinitely, increasing gas costs for `isActiveMarket` lookups and making the unbatched `setOracleBonds` eventually unusable. The batched `setOracleBondsWithBatchCheck` mitigates the DoS but doesn't address the root cause.

Recommendation

Remove markets from `_activeMarkets` after finalization and bond settlement.

Resolution

Trueo: Acknowledged.

I-09 | Bond Amounts Snapshot At Market Creation

Category	Severity	Location	Status
Documentation	Info	TruthMarketManager.sol: 369	Resolved

Description

`resolverBondAmount`, `disputerBondAmount`, and `escalatorBondAmount` are copied from `TruthMarketManager` into each market during `initialize()` and never updated. Admin calls to `TruthMarketManager::setAmounts()` to update these values then only affect future markets. while existing markets always retain their original values for these variables.

Recommendation

Consider documenting that `setAmounts` applies only to future markets, not retroactively.

Resolution

Trueo: Resolved.

I-10 | Single-side LP Helper Allows Zero Liquidity

Category	Severity	Location	Status
Unexpected Behavior	Info	UniswapV4LPHelper.sol	Resolved

Description

`UniswapV4LPHelper.createSingleSidedPositionWithParams` computes and forwards `liquidity` from `getLiquidityForAmounts` without checking for zero:
For small shares or adverse tick geometry, the returned liquidity can be zero even when `amount0`/`amount1` are non-zero.

Recommendation

Short-circuit zero-liquidity cases before calling `modifyLiquidities` (e.g., `if (liquidity == 0) revert` or skip distribution for that depositor with tracked remainder).

Resolution

Trueo: Resolved.

I-11 | Unbounded Ratio Threshold Can Disable Check

Category	Severity	Location	Status
Validation	Info	LaunchpadFactoryCommunityV2.sol, LaunchpadFactoryWhalesV2.sol	Resolved

Description

The proposal spec accepts `maxRatioThreshold` without enforcing an upper bound, while the ratio plugin compares each outcome ratio in basis points against that threshold and returns false only when `ratio > maxRatioThreshold`. Because outcome ratios are computed on a 0 to 10000 scale, any threshold above 10000 makes the check effectively non-binding. In that state, the protocol can treat heavily imbalanced proposals as ready even though the ratio guard is intended to reduce launch fragility near one-sided deposits. This is primarily a configuration-integrity issue. It requires a misconfigured proposal input.

Recommendation

Constrain `maxRatioThreshold` at proposal validation to the expected basis-point domain and reject out-of-range values. A simple rule is `maxRatioThreshold <= 10000`, with explicit handling for any special sentinel semantics such as `0`. Keep the same bound in all proposal composition paths so the ratio guard cannot be silently disabled by configuration.

Resolution

Trueo: Resolved.

I-12 | Max Sqrt Clamp Can Break Tick Conversion

Category	Severity	Location	Status
Validation	Info	UniswapV4LPHelper.sol	Resolved

Description

`priceToSqrtPriceX96` clamps large values to `TickMath.MAX_SQRT_PRICE`. That value is then consumed by `priceToTick`, which calls `TickMath.getTickAtSqrtPrice`. The Uniswap bound for `getTickAtSqrtPrice` is strictly less than `MAX_SQRT_PRICE`, so feeding the clamped max value can revert.
This creates a boundary mismatch between two helper steps that are expected to be compatible. Under extreme ratio and decimal combinations, the clamp path can route execution into an avoidable revert instead of returning a bounded tick. In launch and distribution flows that depend on tick derivation, this can degrade liveness and push proposals into deferred or manual recovery behavior.

Recommendation

Align the clamp with `getTickAtSqrtPrice` preconditions by capping the upper bound at `TickMath.MAX_SQRT_PRICE - 1` before any tick conversion path. Keep the lower bound at `TickMath.MIN_SQRT_PRICE`.

Resolution

Trueo: Resolved.

I-13 | Low Whales Ratio Can Deadlock Launch

Category	Severity	Location	Status
Validation	Info	LaunchpadFactoryWhalesV2.sol	Resolved

Description

composeProposal() normalizes maxRatioThreshold only when it is 0, then forwards the encoded value directly to the RestrictMaximumRatioOfOutcomeDepositPlugin check hook (src/libraries/LaunchpadFactoryWhalesV2.sol:90-99).
The plugin enforces only an upper bound via ratio > maxRatioThreshold and uses the same 2-outcome deposit set that TruthMarketV2Launcher later converts to initial prices. For two-outcome markets, one outcome is always at least 50% by definition, so any configured maxRatioThreshold < 5000 is mathematically unsatisfiable and the proposal can never satisfy launch readiness. totalDeposits >= minimumDeposit can be met, but CheckProposal remains false forever:

Recommendation

Validate maxRatioThreshold in validateSpec() with a lower bound appropriate for two outcomes: If 3+ outcomes are ever supported, define threshold policy by outcome count rather than a hard 2-outcome constant.

Resolution

Trueo: Resolved.

I-14 | Single-side Distributor Reuses Slot0/tick

Category	Severity	Location	Status
Unexpected Behavior	Info	TruthMarketV2SingleSideLiquidityPositionDistributor.sol	Acknowledged

Description

`TruthMarketV2SingleSideLiquidityPositionDistributor` precomputes LP position parameters once per pool and reuses them for every depositor in distribution.
`calculateSingleSidedPositionParams` captures live pool state once:
If pool state is not stable across sequential mints, later depositors can be processed with stale ticks/sqrt price captured from the first call, yielding allocation and slippage behavior inconsistent with current state. This stale snapshot risk is specifically relevant when: • `poolManager.modifyLiquidities(...)` is called repeatedly in one distribution loop and each call changes/realigns `slot0` or `tick` conditions (including protocol-level fee growth, bounds, or derived internal math side effects). • the Uniswap V4 pool is hook-aware (`PoolKey.hooks` is configured at market creation), so hook logic can create non-trivial state effects around each mint action. • minting/ordering effects across the yes/no pools interact indirectly, so parameters become stale for the second and later depositor operations. • distribution is resumed after partial work or retried in...

Recommendation

Add one of the following: • Recompute pool pricing state per depositor (or per bounded batch chunk) before each mint call. • Snapshot + drift-check pattern: • store `slot0` at the start and assert `slot0` unchanged before each subsequent mint; • if changed, pause batch progress and fail-safe with controlled fallback/manual recovery. • Make hook-state assumptions explicit in design/docs only if those assumptions are enforced by code-level constraints. An example fallback pattern can be:

Resolution

Trueo: Acknowledged.

I-15 | Incompatibility With Fee On Transfer Tokens

Category	Severity	Location	Status
Warning	Info	SignatureTransfer.sol	Acknowledged

Description

`Launchpad` records deposits as if the requested `amount` always arrives in escrow. In `deposit`, it updates depositor balances, outcome totals, and the proposal-wide `totalDeposits` before any token transfer takes place. The accounting layer therefore trusts the caller-supplied nominal amount, not the amount that the contract actually receives.
That assumption is only safe if `paymentToken` is a plain ERC20 that always credits the exact requested amount. The contract does not enforce that invariant. If the configured token charges a transfer fee, rebases downward, or has any nonstandard behavior that results in fewer tokens being credited than requested, the proposal accounting immediately becomes overstated. From that point onward the system believes more value is escrowed than actually exists. The Permit2 path does not repair this. It forwards a `safeTransferFrom` call for the requested amount, but it does not compare balances before and after the transfer and it does not prove that escrow received the full amount recorded by `Launchpad`.

Recommendation

If fee on transfer tokens are supported, do not credit proposal accounting from the caller-supplied `amount`. Move the token transfer before the bookkeeping update, or better, measure the actual amount received with a balance-before and balance-after check and credit only that delta. If exact funding is required, revert whenever the received amount is smaller than the requested amount. If the protocol is not intended to support fee-on-transfer or rebasing assets, enforce that operational...

Resolution

Trueo: Acknowledged.

I-16 | Proposal Creator Metadata Is Spoofable

Category	Severity	Location	Status
Warning	Info	Launchpad.sol	Resolved

Description

The `Proposal` struct includes a `creator` field, and `Launchpad` exposes that field back to callers through `proposalCreator`. The problem is that `propose` accepts the entire struct from the caller and stores it without canonicalizing the creator value. The transaction sender must hold `Roles.LAUNCHPAD_PROPOSER_ROLE`, but the stored creator does not have to match that sender. As a result, any proposer-role holder can create a proposal while attributing it to any arbitrary address. The transaction origin is still visible at the chain level, but the protocol's own creator metadata becomes untrustworthy. That matters because a field named `creator`, combined with a dedicated getter, strongly implies that the value is authoritative and safe for user interfaces, attribution logic, allowlists, analytics, revenue-sharing rules, or future policy hooks to consume. Even if the current codebase does not yet gate critical settlement behavior on `proposal.creator`, exposing spoofable identity metadata is still a protocol footgun. Off-chain systems can display the wrong responsible party, an...

Recommendation

Do not take `creator` from user input. Set the stored creator to `msg.sender` inside `propose`, or remove the field from the external proposal payload entirely and derive it internally. If delegated creation is a real product requirement, separate the concepts of proposer and creator instead of overloading one field. In that model, the contract should record `msg.sender` as the proposer and only accept a different creator when it is backed by explicit authorization, such as a signature from the...

Resolution

Trueo: Resolved.

Round Two Findings & Resolutions

ID	Title	Category	Severity	Status
M-01	Launch Failures Trap Ready Proposals	DoS	Medium	Resolved
M-02	Expired Proposals Can Still Launch Late	Validation	Medium	Resolved
M-03	Owner Council Overrides Can Brick Settlement	DoS	Medium	Resolved
M-04	OracleBonds Migration Breaks Late Bond Claims	Unexpected Behavior	Medium	Acknowledged
M-05	Packed Slots Keep Historical Crossing Cost	DoS	Medium	Resolved
M-06	Upward Boundary Touch Can Miss Fills	Logical Error	Medium	Resolved
L-01	Launchpad Markets Record Launcher As Creator	Warning	Low	Resolved
L-02	Distributor Rounding Dust Stays Stranded	Rounding	Low	Resolved
L-03	Community LP Payouts Are Immediately Unwindable	Warning	Low	Acknowledged
L-04	Trading End Boundary Mismatch	Validation	Low	Resolved
L-05	Duration Changes Can Break Launch	Warning	Low	Acknowledged
I-01	Launchpad Accepts Incompatible Custom Proposals	Warning	Info	Partially Resolved
I-02	Launch Can Finalize Without Active Liquidity	Warning	Info	Acknowledged

M-01 | Launch Failures Trap Ready Proposals

Category	Severity	Location	Status
DoS	Medium	Launchpad.sol	Resolved

Description

Launchpad.withdraw() still calls _attemptLaunch() after recalculating balances and Launchpad._attemptLaunch() calls ILauncher.launch(...) directly with no try/catch and no refundable failure state. If a proposal remains launch-ready after a small withdrawal and launch() reverts, the whole withdrawal reverts too. This is reachable on the canonical factory path because proposal validation is still weaker than runtime launch validation.
LaunchpadFactoryMarketV2.validateCommon() only checks non-empty question, non-empty outcome symbols and the trading window, while TruthMarketManager.createMarketV2() and _createMarketCommon() can still revert on empty marketSource, invalid or duplicate symbols, invalid fees, or bad V2 manager configuration. Once such a proposal becomes ready, every deposit or withdrawal that still leaves it ready will retry launch() and revert again. Users can only get their funds out once the proposal expires into Failed, so smaller holders can be effectively frozen until endTime. The factory enforces only a minimum proposal duration, not a...

Recommendation

Wrap ILauncher.launch(...) in try/catch and move the proposal into a refundable failure phase when launch fails, instead of retrying the same bad launch on every later interaction. Proposal creation should also validate the same launcher and manager preconditions that runtime launch enforces, or ask the launcher path to perform a dry-run validation before the proposal is accepted. If long-lived proposals are not intended, add a maximum proposal duration.

Resolution

Trueo: Resolved.

M-02 | Expired Proposals Can Still Launch Late

Category	Severity	Location	Status
Validation	Medium	Launchpad.sol	Resolved

Description

resolveDeferredLaunch() checks only the stored state.phase before retrying launch. It does not recompute the effective phase with _proposalPhase(), so it can retry launch from stale Live state after expiry. deposit() and withdraw() are safe because they recompute the phase and persist the transition with _handlePhaseTransition() before calling _attemptLaunch(). resolveDeferredLaunch() skips that step and calls _attemptLaunch() directly. _attemptLaunch() then relies on the same stale stored state.phase and does not perform its own expiry check. This lets an expired proposal launch after the fundraising deadline as long as it is still stored as Live. The most realistic case is a proposal that already satisfied its launch conditions before expiry, but a prior launch attempt failed and left the proposal in Live. After endTime, the UI and viewer report Failed, so users reasonably expect refunds. Instead, any caller can invoke resolveDeferredLaunch() and commit the escrowed funds into a late market launch once the launcher stops reverting. The default...

Recommendation

Recompute the phase inside resolveDeferredLaunch() before retrying launch and persist the transition with _handlePhaseTransition(). If the computed phase is no longer Live, revert instead of calling _attemptLaunch(). As a defense in depth measure, _attemptLaunch() can also reject proposals whose computed phase is not Live.

Resolution

Trueo: Resolved.

M-03 | Owner Council Overrides Can Brick Settlement

Category	Severity	Location	Status
DoS	Medium	TruthMarketManager.sol	Resolved

Description

TruthMarketManager lets its owner call resolveMarketByCouncil() and resetMarketByCouncil() directly. Those functions only forward into the market contract. They do not create or close an OracleCouncil dispute first. TruthMarket and TruthMarketV2 treat those calls as if council bookkeeping already exists. resolveMarketByCouncil() records councilDecisionAt, which later makes _settleBonds() read oracleCouncil.getLastClosedDispute(address(this)). resetMarketByCouncil(true) makes the same assumption immediately and resetMarketByCouncil(false) reaches it on the next proposeResolution(). If the owner used the manager override instead of letting OracleCouncil._closeDispute() drive the transition, marketLastClosedDispute[_market] is still zero. Therefore getLastClosedDispute() returns the zero-initialized dispute at index 0. That zero struct carries disputorAddress == address(0). The market then calls oracleBonds.issueBondsBackToDisputor(address(this), address(0)) and OracleBonds._transferBondFromMarket() reverts on the zero-address check. Consequently...

Recommendation

Remove direct owner access to the council-only override functions, or require proof of a real closed dispute before either override can execute. The safer fix is to pass an explicit dispute index from OracleCouncil and reject the call unless that dispute exists and belongs to the market. TruthMarket and TruthMarketV2 should not infer a valid disputor from councilDecisionAt alone and should never call issueBondsBackToDisputor() with a zero address.

Resolution

Trueo: Resolved.

M-04 | OracleBonds Migration Breaks Late Bond Claims

Category	Severity	Location	Status
Unexpected Behavior	Medium	TruthMarketManager.sol	Acknowledged

Description

TruthMarketManager.setOracleBonds() and setOracleBondsWithBatchCheck() allow the global bond contract to be replaced as soon as every active market reports bondSettled == true. That gate is too weak. In both TruthMarket and TruthMarketV2, _settleBonds() only settles the resolver bond and the last closed dispute or escalation outcome, then marks the market as settled. It does not clear disputor bonds that remain claimable through OracleCouncil.claimUnclosedDisputeBonds(). The problem is that the late-claim logic in OracleCouncil always resolves against the current marketManager.oracleBonds() address. It does this both in canDisputorClaimbackBondFromUnclosedDispute() and in claimUnclosedDisputeBonds(). Existing end-to-end tests already rely on this delayed claim behavior after redeem() has triggered bond settlement. If the owner migrates oracleBonds before that claim happens, the new OracleBonds contract has no legacy marketBond state for the old market, so the claim check fails and the user cannot recover funds through the normal interface. The real bond...

Recommendation

Do not use bondSettled as the migration gate for oracleBonds. Before switching addresses, require the old OracleBonds contract to have no remaining claimable balance for each active market, including disputor balances for unclosed disputes. If migration must happen while old markets still have pending claims, store and use a market-specific historical OracleBonds address for claim functions, or migrate the per-market bond state into the new contract atomically.

Resolution

Trueo: Acknowledged.

M-05 | Packed Slots Keep Historical Crossing Cost

Category	Severity	Location	Status
DoS	Medium	OrderBook.sol	Resolved

Description

`removeOrder()` decrements `orderCounts[tickThreshold]` and clears the removed packed position, but it never shrinks `orders[tickThreshold]` when trailing packed slots become empty. This leaves the packed array length tied to the largest number of orders that ever existed at that tick instead of the number still live. That stale length is later trusted by both `countPackedOrders()` and `moveTick()`, which use `self.orders[next].length` to count and copy packed slots. Consequently, crossing cost depends on the historical peak occupancy of a tick, not the current live order count. An attacker can abuse this cheaply. They can create a large number of orders at one tick, then cancel all but one. The bitmap still marks the tick as initialized because one order remains. However, crossing that tick still processes the old packed-slot array instead of the one live order. This turns a short-lived burst of order spam into a persistent liveness problem and makes the crossed-order gas barrier much cheaper to maintain.

Recommendation

Keep the packed array length consistent with the live logical count. Consider shrinking trailing empty packed slots during `removeOrder()`. If that is too expensive, stop using `orders[tick].length` as the authoritative packed length and derive the live packed-slot count from `orderCounts[tick]` instead.

Resolution

Trueo: Resolved.

M-06 | Upward Boundary Touch Can Miss Fills

Category	Severity	Location	Status
Logical Error	Medium	OrderManager.sol	Resolved

Description

`_isTickInRange()` uses a half-open interval for upward moves: `fromTick <= tick && tick < toTick`. `OrderManager` uses that predicate for both full fills and partial fills on `zeroForOne` orders. Therefore a threshold is treated as crossed only when it is strictly below the post-swap `toTick`. This becomes wrong when a swap ends exactly on the threshold. `TruthMarketHook._afterSwap()` reads the authoritative post-swap `slot0.tick` and Uniswap stores the exact upper boundary tick on upward moves. If `toTick` equals `tickUpper` or `tickLower + tickSpacing`, `_isTickInRange()` excludes that threshold even though the order has already reached the trigger boundary. The order stays pending until price moves one more tick upward. Price can reverse before that happens, so the user remains exposed after their limit was touched. This affects upward execution checks for `zeroForOne` orders. The issue is visible in both the full-fill check and the partial-fill check because they both rely on the same predicate.

Recommendation

Make upward threshold checks treat an exact post-swap boundary touch as crossed. The smallest safe fix is to adjust the upward comparison used for execution thresholds so the upper bound is inclusive for these checks or to normalize `toTick` before calling `_isTickInRange()` when the pool ends exactly on a trigger boundary.

Resolution

Trueo: Resolved.

L-01 | Launchpad Markets Record Launcher As Creator

Category	Severity	Location	Status
Warning	Low	TruthMarketManager.sol	Resolved

Description

`TruthMarketManager._createMarketCommon()` records `creatorAddress[marketAddress] = msg.sender` and emits `MarketCreatedWithDescription(..., marketOwner = msg.sender)`. Under the launchpad flow, `msg.sender` at that point is the `TruthMarketV2Launcher` contract, because `Launchpad` calls the launcher and the launcher calls `createMarketV2()`. The human proposal creator tracked by the launchpad is not propagated into manager-level market creation. As a result, launchpad-created markets end up with incorrect creator metadata at the market-manager layer even though the proposal creator is already available through `ILaunchpadViewer.proposalCreator(proposalId)`. In the current scoped code this is mostly an attribution and accounting bug, but it becomes more serious if any downstream logic, analytics, or fee-routing integration treats `creatorAddress` or the emitted `marketOwner` as the canonical market creator. In that case, creator-linked rewards or revenue could be misattributed to the launcher contract instead of the actual proposer.

Recommendation

Do not derive the market creator from `msg.sender` in the launchpad path. Pass an explicit canonical creator into market creation and store that value in `creatorAddress` and the creation event. If the launchpad's `proposal.creator` field is used as the source, first ensure that field is itself canonicalized to the real proposer instead of trusting arbitrary calldata.

Resolution

Trueo: Resolved.

L-02 | Distributor Rounding Dust Stays Stranded

Category	Severity	Location	Status
Rounding	Low	TruthMarketV2ProportionalDistributor.sol, TruthMarketV2SingleSideLiquidityPositionDistributor.sol	Resolved

Description

TruthMarketV2ProportionalDistributor mints the full YES and NO balances to itself and then distributes them with per-user integer division. totalTokensMinted is computed once, but each depositor receives tokenAmount = (totalTokensMinted * depositAmount) / totalOutcomeDeposits. Because each user amount is rounded down independently, the sum of all transfers can be smaller than the minted balance. Any remainder stays on the distributor contract. The same issue exists in TruthMarketV2SingleSideLiquidityPositionDistributor, where each depositor's share is rounded down once when computing distributableAmount and again when converting that amount into outcome tokens before transfer and LP minting. I do not see any sweep, rescue, or deterministic remainder handling in either distributor, so leftover YES and NO tokens can remain stranded on the distributor contracts indefinitely. The value is usually small, but it is permanent and can accumulate across proposals.

Recommendation

Handle remainders explicitly instead of leaving them on the distributor. The cleanest fix is to track how many YES and NO tokens were actually distributed and then route the leftover balance at the end of distribution to a deterministic sink, or distribute the final remainder to a designated recipient under documented rules. If dust is intentionally tolerated, add a restricted sweep function so stranded balances are recoverable.

Resolution

Trueo: Resolved.

L-03 | Community LP Payouts Are Immediately Unwindable

Category	Severity	Location	Status
Warning	Low	TruthMarketV2SingleSideLiquidityPositionDistributor.sol	Acknowledged

Description

In the community distribution path, a depositor does not end up with a one-sided post-launch position. `TruthMarketV2SingleSideLiquidityPositionDistributor._processDepositorOutcome()` transfers the depositor's chosen-side outcome tokens directly to them and then mints an opposing-side LP NFT directly to the same depositor. There is no lock, escrow, or vesting step on that NFT. Once `TruthMarketV2Launcher.finalize()` unpauses the pools, the user can use the standard position-manager flow to remove liquidity from that NFT and recover the opposing-side outcome tokens. That leaves the user holding both YES and NO in matched size. `TruthMarketV2.burn()` and `TruthMarket.burn()` then let the holder burn equal YES and NO amounts back into payment tokens before market finalization. In practice, a community participant can help set the launch ratio, wait for distribution, unwind the opposing-side LP and convert most of the position back into payment tokens. Their net cost becomes mostly protocol fee, liquidity-removal friction and gas, rather than the full directional exposure implied by the...

Recommendation

If the community flow is meant to represent committed directional exposure, do not hand the opposing-side LP position to the depositor in an immediately removable form. A straightforward fix is to lock community LP NFTs for a minimum period or mint them to an escrow contract that enforces delayed withdrawal. If immediate user custody is required, the docs and product assumptions should explicitly treat the launch ratio as a weak signaling mechanism rather than committed post-launch exposure.

Resolution

Trueo: Acknowledged.

L-04 | Trading End Boundary Mismatch

Category	Severity	Location	Status
Validation	Low	LaunchpadFactoryMarketV2.sol, TruthMarketManager.sol	Resolved

Description

The factory requires endOfTrading > endOfProposal + minimumTradingDuration, while the runtime market creation check only requires endOfTrading >= block.timestamp + minimumTradingDuration. This creates a boundary inconsistency where proposals using an exact-minimum trading window are rejected by the factory even though the manager would accept them at launch. The issue does not create unsafe launches, but it does make proposal validation stricter than the runtime policy and can cause unnecessary proposal rejection.

Recommendation

Align the comparators used by the factory and manager so both stages enforce the same boundary condition.

Resolution

Trueo: Resolved.

L-05 | Duration Changes Can Break Launch

Category	Severity	Location	Status
Warning	Low	LaunchpadFactoryMarketV2.sol, TruthMarketManager.sol	Acknowledged

Description

The initial factory-side validation now correctly fetches the current minimumTradingDuration from the launcher instead of relying on a hardcoded threshold. However, launch-time market creation re-validates endOfTrading against TruthMarketManager.minimumTradingDuration, which remains mutable through setDurations(). As a result, proposals accepted under one duration can still become unlaunchable if governance increases that value before launch.

Recommendation

Be aware that increasing minimumTradingDuration can invalidate already-accepted proposals that have not launched yet. When changing this value, pending proposals should be reviewed to determine whether they may become unlaunchable under the new duration. If minimumTradingDuration may be changed frequently, consider snapshotting the effective value at proposal creation so launch-time validation uses the same duration that was used when the proposal was accepted.

Resolution

Trueo: Acknowledged.

I-01 | Launchpad Accepts Incompatible Custom Proposals

Category	Severity	Location	Status
Warning	Info	Launchpad.sol	Partially Resolved

Description

`Launchpad.propose()` does not validate that a custom proposal is actually compatible with the binary `TruthMarketV2` launch and distribution flow. It only requires at least one `CheckProposal` hook and the presence of a `BeforeDeposit` `PLUGIN_RESTRICT_OUTCOME` hook. It does not verify that the outcome whitelist is exactly `[1,2]` and it does not require the minimum-deposit or max-ratio safety hooks that the canonical factories always attach. This makes the core contract rely on factory-only invariants. A proposer-role holder can bypass the factory and submit a proposal with a wider outcome whitelist or a do-nothing readiness hook set. That is enough to accept deposits that downstream code never handles. `TruthMarketV2Launcher` and both distributors still read and distribute only outcomes `1` and `2`, so deposits into any additional outcome can remain counted in `proposalTotalDeposits` while never being paid out. The same gap also allows custom proposals to omit readiness hooks that prevent one-sided or otherwise unlaunchable states. `RestrictOutcomePlugin.checkParameters()` does...

Recommendation

Enforce launcher and distributor compatibility in the core proposal path instead of trusting factory templates. For the current binary design, require the outcome restriction args to decode to exactly `[1,2]` and reject proposals that do not include the safety hooks expected by the selected launcher and distributor pair. `RestrictOutcomePlugin.checkParameters()` should also validate that its whitelist matches the binary-only invariant.

Resolution

Trueo: Partially Resolved.

I-02 | Launch Can Finalize Without Active Liquidity

Category	Severity	Location	Status
Warning	Info	TruthMarketV2Launcher.sol	Acknowledged

Description

`TruthMarketV2Launcher.launch()` initializes pool price but does not seed any in-range liquidity and `finalize()` only unpauses the pools. The two distributor paths do not close that gap. `TruthMarketV2ProportionalDistributor` gives whales users tokens only, with no LP position at all. `TruthMarketV2SingleSideLiquidityPositionDistributor` does mint LP NFTs for community users, but `UniswapV4LPHelper.calculateSingleSidedPositionParams()` places those positions strictly above or below the current tick, so they are intentionally out of range at launch rather than immediately tradable liquidity. This means a market can reach `Distributed`, be finalized and still open with zero active liquidity on either side. In that state, the first external actor willing to supply in-range liquidity becomes the effective launch-time market maker. That actor can dominate the initial spread, capture most early fees and decide how easy it is for freshly distributed token holders to exit or hedge. If no one steps in to add active liquidity, the market is technically launched but not meaningfully tradable....

Recommendation

If immediate trading is part of the product promise, seed protocol-controlled in-range liquidity during launch or finalization instead of relying on third parties to bootstrap the book after the fact. If that is not intended, the docs and UI should describe launch as token distribution first and market making later, so users are not led to assume that a finalized market is automatically liquid and ready for fair price discovery.

Resolution

Trueo: Acknowledged.

Round Three Findings & Resolutions

ID	Title	Category	Severity	Status
H-01	Hook Batching Starts After Full Order Scan	DoS	High	Resolved
M-01	Council Or Escalation Rotation Can Brick Markets	DoS	Medium	Acknowledged
M-02	Payment Token Drift Can Brick V2 Launches	DoS	Medium	Resolved
L-01	Launchpad Finalize/fee Reverts Bypass Fallback	DoS	Low	Resolved
L-02	Launchpad V2 Misses Swap-validator Bounds	Unexpected Behavior	Low	Resolved
L-03	Launchpad V2 Accepts Invalid Tick Spacing	Validation	Low	Resolved
L-04	Per-deposit Minimum Can Truncate To Zero	Rounding	Low	Resolved
L-05	Dust Partial Fills Requeue Deleted Orders	Logical Error	Low	Resolved
I-01	Launchpad Fee Is Not Snapshotted Per Proposal	Trust Assumptions	Info	Acknowledged

H-01 | Hook Batching Starts After Full Order Scan

Category	Severity	Location	Status
DoS	High	OrderManager.sol	Resolved

Description
`movePoolTick()` calls `orderBook.moveTick()` before enforcing `maximumExecutionCount`. That defeats the intended gas cap because `moveTick()` is already the expensive step: it traverses the crossed range, materializes the full crossed set in memory, and deletes those ticks from storage before deferral is considered.
This matters because `TruthMarketHook._afterSwap()` executes the logic inside the swap callback. A trader can concentrate enough live orders around a target range that every attempt to cross it pays the same unbounded preprocessing cost and can run out of gas before the callback completes.

Recommendation
Apply the execution budget before materializing the full crossed range. Process only a bounded slice of ticks or packed orders per callback and persist a continuation cursor for resolver transactions.

Resolution
Trueo: Resolved.

M-01 | Council Or Escalation Rotation Can Brick Markets

Category	Severity	Location	Status
DoS	Medium	TruthMarketManager.sol	Acknowledged

Description
`TruthMarketManager.setAddresses()` can rotate `oracleCouncilAddress` and `escalationAddress`, but existing markets do not follow those changes. They cache the original contracts at initialization and continue reading dispute state from the old dependencies.
After rotation, the manager authorizes only the new council or escalation contracts, while live markets can still look to the old ones for dispute records. That split can leave later dispute settlement reading stale or zero data and make the market impossible to settle without manual restoration of the historical dependencies.

Recommendation
Do not rotate these addresses while any market can still dispute, escalate, reset, or settle bonds. The safer fix is to resolve dependencies dynamically from the manager or treat them as immutable per market.

Resolution
Trueo: Acknowledged.

M-02 | Payment Token Drift Can Brick V2 Launches

Category	Severity	Location	Status
DoS	Medium	TruthMarketManager.sol	Resolved

Description
`TruthMarketManager.setAddresses()` can change the global `paymentToken`, but `Launchpad` and the V2 distributors keep the token that was wired when they were deployed or initialized. That creates a split configuration where deposits are escrowed in the old token while newly created V2 markets expect the manager's new token.
In that state, proposals can accept deposits successfully and fail only after launch has started, pushing settlement into deferred or manual handling.

Recommendation
Do not let the manager payment token drift independently from the launchpad stack. Treat it as immutable for a deployed stack, or redeploy and rewire the full stack atomically when it changes.

Resolution
Trueo: Resolved.

L-01 | Launchpad Finalize/fee Reverts Bypass Fallback

Category	Severity	Location	Status
DoS	Low	Launchpad.sol	Resolved

Description
`Launchpad._distribute()` only wraps the distributor call in `try/catch`. If the later fee transfer or `finalize()` call reverts, that failure bypasses the fallback path entirely.
When that happens, `failedDistributionAttempts` is not incremented and the proposal never progresses toward `ManualDistributionNeeded`. The same revert simply repeats on every retry until an operator changes the external configuration.

Recommendation
Wrap the full post-distribution sequence, not just the distributor call, in the same failure-accounting path so all revert cases are counted and can eventually route to manual handling.

Resolution
Trueo: Resolved.

L-02 | Launchpad V2 Misses Swap-validator Bounds

Category	Severity	Location	Status
Unexpected Behavior	Low	TruthMarketV2SingleSideLiquidityPositionDistributor.sol	Resolved

Description
The LP-manager flow initializes swap-validator boundaries before minting V2 liquidity, but the launchpad V2 flow does not. If a `TruthMarketHook` swap validator is configured, launchpad-created markets can therefore run with default full-range bounds instead of the intended binary-price envelope.
That means equivalent V2 markets can have different protections depending on how they were launched, and operational assumptions about shared swap-boundary behavior become false.

Recommendation
Initialize swap-validator bounds in one canonical V2 launch path and fail closed if a market reaches distribution or finalization without the required boundaries set.

Resolution
Trueo: Resolved.

L-03 | Launchpad V2 Accepts Invalid Tick Spacing

Category	Severity	Location	Status
Validation	Low	LaunchpadFactoryMarketV2.sol	Resolved

Description
`LaunchpadFactoryMarketV2.validateCommon()` checks fee and related fields, but it does not validate that `tickSpacing` is positive and within Uniswap V4 bounds. Invalid values can therefore be accepted into a proposal and only fail later when pool initialization is attempted.
That turns a known-bad configuration into a deferred launch-time failure instead of rejecting it up front.

Recommendation
Validate `tickSpacing` before proposal acceptance or market creation. At minimum, reject values `<= 0` and values above the supported Uniswap V4 maximum.

Resolution
Trueo: Resolved.

L-04 | Per-deposit Minimum Can Truncate To Zero

Category	Severity	Location	Status
Rounding	Low	LaunchpadFactoryCommunityV2.sol	Resolved

Description
The community and whales factories derive per-deposit thresholds as `minimumDeposit / 100`. Because this uses integer division, any configured minimum below `100` raw token units truncates to `0`.
That weakens the intended dust-deposit and dust-remainder protections, since the hooks then accept deposits and withdrawals that were supposed to fall below the enforced floor.

Recommendation
Do not allow the derived threshold to become zero. Either require `minimumDeposit >= 100` or clamp the hook input to a nonzero floor such as `max(1, minimumDeposit / 100)`.

Resolution
Trueo: Resolved.

L-05 | Dust Partial Fills Requeue Deleted Orders

Category	Severity	Location	Status
Logical Error	Low	OrderManager.sol	Resolved

Description
`_executeOrders()` treats any `false` return from `_executeOrder()` as “order not executed” and requeues the order ID. That is incorrect for dust-settled partial fills.
When `_partialFillOrder()` fully settles the remaining dust and deletes the pending order, `_executeOrder()` still returns `false`, so `_executeOrders()` can reinsert an order ID whose storage entry no longer exists.

Recommendation
Return an explicit execution status instead of using one boolean for both “not executed” and “executed but closed.” Requeue an order only if it still exists after execution.

Resolution
Trueo: Resolved.

I-01 | Launchpad Fee Is Not Snapshotted Per Proposal

Category	Severity	Location	Status
Trust Assumptions	Info	Launchpad.sol	Acknowledged

Description
`setProtocolFee()` and `setFeeReceiver()` can change the fee regime at any time, but `Launchpad` does not snapshot either value per proposal. Withdrawals, launch-time distribution, and manual distribution all use the current global fee settings when execution happens.
As a result, users can deposit under one fee regime and settle under another. That makes the effective fee on a live proposal a governance decision at settlement time rather than a property fixed when users entered.

Recommendation
If fee terms should be stable for live proposals, snapshot the fee rate and receiver at `propose()` time and use those values for later settlement. Otherwise, document the behavior clearly and constrain fee changes operationally with a timelock or similar process.

Resolution
Trueo: Acknowledged.

Round Four Findings & Resolutions

ID	Title	Category	Severity	Status
M-01	Deferred Dense Tick Can Become Unresolvable	DoS	Medium	Resolved
M-02	Deferred Replay Can Duplicate Partial Thresholds	DoS	Medium	Resolved
L-01	Ownership Transfer Leaves Stale Role Admins	Trust Assumptions	Low	Resolved
L-02	Hookless V2 Launches Skip The Pause Guard	Validation	Low	Resolved

M-01 | Deferred Dense Tick Can Become Unresolvable

Category	Severity	Location	Status
DoS	Medium	OrderManager.sol	Resolved

Description
The tick-range deferral fix pages across ticks, but it still assumes any single tick can always be processed in one transaction. That assumption is unenforced: there is no per-tick order cap.
If enough orders accumulate on one threshold, the resolver must still copy, delete, and execute the entire dense tick in one call. That transaction can exceed block gas limits and revert permanently, leaving the affected orders stranded unless operators unwind them manually.

Recommendation
Do not rely on whole-tick execution as the final liveness guarantee. Enforce a hard per-tick order cap that stays comfortably below worst-case gas limits.

Resolution
Trueo: Resolved.

M-02 | Deferred Replay Can Duplicate Partial Thresholds

Category	Severity	Location	Status
DoS	Medium	OrderManager.sol	Resolved

Description
`_executeOrders()` requeues partial-fill thresholds based on the original deferred tick range, assuming `moveTick()` removed every matching threshold. That assumption fails once deferred processing is split across bounded tick batches.
If the current batch did not actually remove one of the thresholds, `_executeOrders()` can insert it again. Repeated retries can therefore grow duplicate threshold entries, increase tick density, and eventually turn deferred resolution itself into a gas-driven liveness failure.

Recommendation
Requeue only thresholds that the current `moveTick()` batch actually removed. Pass the processed tick interval or exact removed set into `_executeOrders()` instead of using the original deferred range as a proxy.

Resolution
Trueo: Resolved.

L-01 | Ownership Transfer Leaves Stale Role Admins

Category	Severity	Location	Status
Trust Assumptions	Low	TruthMarketManager.sol	Resolved

Description
`TruthMarketManager` mixes `Ownable` and `AccessControl`, but `transferOwnership()` only updates the owner slot. It does not revoke or migrate `DEFAULT_ADMIN_ROLE` or `PAUSER_ROLE`.
As a result, a former owner can retain meaningful control after an apparent governance handoff, including the ability to regrant privileged roles or pause and unpause the system.

Recommendation
Do not leave ownership transfer and role administration decoupled. Ownership handoff should also rotate the critical roles, or the protocol should enforce a formal handoff procedure that completes both steps together.

Resolution
Trueo: Resolved.

L-02 | Hookless V2 Launches Skip The Pause Guard

Category	Severity	Location	Status
Validation	Low	TruthMarketV2Launcher.sol	Resolved

Description
`TruthMarketV2Launcher.launch()` only pauses the new pools when both pools share a nonzero compatible hook. A hookless configuration is still treated as valid and returns success instead of reverting.
That breaks the deferred-settlement safety assumption that newly launched V2 markets remain inert until distribution succeeds. If settlement is deferred, a hookless market can already be live and interactive while depositors are still unpaid.

Recommendation
Reject launchpad V2 launches when the manager has no compatible pause-capable hook configured. The pause guarantee should be mandatory, not optional.

Resolution
Trueo: Resolved.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit https://guardianaudits.com

To view our audit portfolio, visit https://github.com/guardianaudits

To book an audit, message https://t.me/guardianaudits